PYTHON LEARNER'S GUIDE

Learn Programming by Reading Real Code

A Complete Companion to the **Customer Churn Intelligence Platform**

12 Chapters	**50+ Code Examples**	**Real Analogies**	**Before & After**	**Common Mistakes**

Variables & Functions	**Pandas & NumPy**	**Machine Learning**	**Error Handling**	**Best Practices**

Designed for learners with basic Python knowledge · Builds to intermediate level · No ML background required

CH 00

How to Read This Guide

Start here before Chapter 1

This guide teaches Python by walking through every concept found in one real, complete, production-quality codebase: the Customer Churn Intelligence Platform. Instead of toy examples, you see concepts exactly where they appear in working code that solves a real business problem.

What This Guide Covers

Each chapter focuses on one major Python concept. By the end you will understand:

- How Python programmes are structured from top to bottom
- How variables, functions, and data structures are used in practice
- How real data cleaning code handles messy, broken input
- How machine learning models are trained and evaluated in 10 lines
- How professional Excel reports are generated programmatically
- How to write code that does not crash on bad data

The Visual Language

Every concept is explained using a consistent set of visual boxes:

ANALOGY — This is an analogy — a comparison from everyday life that makes a concept 'click'. Read these first before looking at the code.

CONCEPT — This is a concept box — it gives the precise technical definition after the analogy has prepared your mind for it.

KEY POINT — This is a key point — the single most important thing to remember about this topic.

TIP — This is a tip — practical advice that makes your code better or saves you time.

MISTAKE — This is a common mistake box — things beginners often get wrong, so you can avoid them.

CAUTION — This is a caution box — behaviour that can surprise you or cause subtle bugs.

REMEMBER — This is a remember box — important details that are easy to forget.

How Code Blocks Are Shown

All code is shown in numbered blocks so you can reference specific lines:

```
example.py
1  # Lines starting with # are COMMENTS — Python ignores them
2  # They explain what the code does, for human readers
3  name = 'Abhishek'   # ← this line is highlighted as important
4  print(name)          # output: Abhishek
5  # Highlighted lines are the ones worth paying special attention to
```

Green text = comments. Highlighted rows = lines to focus on.

CH 01

Programme Structure & How Python Runs

The skeleton of a Python script

Before writing a single line of useful code, you need to understand how a Python file is organised and executed. The Churn Platform follows a classic single-file script structure.

The Six Zones of a Python Script

Every well-written Python script follows this top-to-bottom layout. Reading a new codebase becomes much easier once you recognise these zones:

Zone	Name	What lives here
1	`Imports`	import os, import pandas as pd — libraries you need
2	`Constants`	MAX_RETRIES = 3 — fixed values used throughout the script
3	`Helper fns`	Small utility functions like norm(), safe_float(), clean_num()
4	`GUI setup`	root = tk.Tk() — launch the interface (hides immediately)
5	`Main logic`	The big try/except block — load file → clean → ML → report
6	`Error handler`	except Exception: show error dialog and clean up

Imports — Borrowing Code from Libraries

ANALOGY

Python imports are like a toolbox. Your script is the workbench. Instead of building every tool from scratch, you pull in a screwdriver (pandas), a measuring tape (numpy), and a saw (sklearn) from shelves labelled with the library name.

imports_explained.py

```python
# STANDARD LIBRARY — built into Python, no install needed
import os           # file paths: os.path.basename(), os.path.splitext()
import re           # regular expressions: re.sub(), re.search()
import sys          # system: sys.exit() to stop the programme
import time         # timing: time.sleep(), time.strftime()
import warnings     # suppress noisy library output
import traceback    # full error stack traces for debugging

# THIRD-PARTY — installed via pip
import numpy  as np   # fast number arrays (aliased to 'np')
import pandas as pd   # data tables     (aliased to 'pd')

# FROM a library, import specific classes/functions
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics  import accuracy_score, roc_auc_score
```

KEY POINT

The 'as' keyword creates an alias. 'import numpy as np' means: everywhere I would type 'numpy.something', I can type 'np.something' instead. This is a universal Python convention — always use np for numpy and pd for pandas.

Constants — Fixed Values at the Top

Constants are values that will not change while the programme runs. Placing them at the top makes them easy to find and change in one place.

constants.py

```
# ============================================================
#  CONSTANTS — change these to tune the programme's behaviour
# ============================================================
MAX_RETRIES     = 3    # how many times to retry a locked file
SHEET_NAME_LEN  = 31   # Excel's hard limit on sheet name length
LABEL_ENC_MAX   = 20   # max unique text values before we drop a column
MIN_ROWS_FOR_ML = 20   # minimum dataset size to attempt ML training
```

TIP

Name constants in ALL_CAPS — it signals to every reader: 'I chose not to change this'.
Place ALL constants at the very top of your file, before any functions.

The Main try/except Block

The entire 'main' logic of the Churn Platform is wrapped in a single try/except block at module level. This is an important pattern: one top-level safety net catches any crash and shows a friendly error dialog instead of a Python traceback.

main_structure.py

```
try:
    # ── All the real work happens here ──────────────────────
    file_path = ask_file()           # step 1: user picks file
    raw_df    = load_file(file_path) # step 2: load data
    df        = clean(raw_df)        # step 3: clean data
    clf       = train_model(df)      # step 4: ML model
    build_report(df, clf, save_path) # step 5: write Excel

except SystemExit:
    pass   # user cancelled — not an error, just exit cleanly

except Exception as e:
    # Catch everything else and show a human-readable message
    err('Error', str(e) + traceback.format_exc()[-800:])

finally:
    root.destroy()   # ALWAYS close the GUI, even on crash
```

CH 02

Variables & Data Types

Storing and organising information

Every piece of information in a Python programme lives in a variable. Understanding Python's data types tells you what operations are valid on each piece of data.

Assignment & Naming Rules

variables.py

```
# Assign with =
churn_rate   = 26.5          # float (decimal number)
total_cust   = 7043          # int   (whole number)
model_name   = 'Random Forest' # str  (text)
is_synthetic = True          # bool  (True or False)
missing_val  = None          # NoneType (absence of value)

# Multiple assignment in one line
accuracy, auc = 0.812, 0.874

# Augmented assignment
total_cust += 1   # same as: total_cust = total_cust + 1
```

Type	Description & Example
`int`	Whole numbers: 42, -7, 1_000_000 (underscores ok as separators)
`float`	Decimal numbers: 3.14, -0.5, 1.2e6 (scientific notation)
`str`	Text in single or double quotes: 'hello', "world"
`bool`	True or False only — note capital T and F
`None`	The absence of a value — like null in other languages
`list`	Ordered, changeable sequence: [1, 'two', 3.0]
`dict`	Key-value pairs: {'name': 'Alice', 'age': 30}
`tuple`	Ordered, UNCHANGEABLE sequence: (1, 2, 3)
`set`	Unordered, unique values: {1, 2, 3}
`frozenset`	Like set but IMMUTABLE (cannot be changed after creation)

Type Conversion

Python will not automatically convert between types. You must do it explicitly:

type_conversion.py

```
int('42')          # → 42          string → integer
```

```
float('3.14')       # → 3.14        string → float
str(42)             # → '42'        integer → string
bool(0)             # → False       zero is falsy
bool(1)             # → True        any non-zero is truthy
bool('')            # → False       empty string is falsy
bool('hello')       # → True        non-empty string is truthy

# pd.to_numeric — safe conversion with errors='coerce'
pd.to_numeric('bad text', errors='coerce')  # → NaN (not a crash!)
pd.to_numeric('42.5',     errors='coerce')  # → 42.5
```

CAUTION

pd.to_numeric(errors='coerce') is your best friend for real-world data. Raw datasets often have text in numeric columns. 'coerce' converts what it can and turns everything else into NaN (not a number) instead of crashing.

f-strings — The Modern Way to Build Strings

An f-string (formatted string literal) lets you embed Python expressions directly inside a string. It is the cleanest and most readable way to build text.

fstrings.py

```
name        = 'Abhishek'
churn_rate = 26.4789
revenue     = 1234567.89
customers   = 7043

# Basic embedding
f'Hello, {name}!'                       # → 'Hello, Abhishek!'

# Format spec after the colon
f'{churn_rate:.2f}%'                    # → '26.48%'    (2 decimal places)
f'{churn_rate:.1%}'                     # → '2647.9%'   (as percentage * 100)
f'Rs.{revenue:,.0f}'                    # → 'Rs.12,34,567'   (comma-separated)
f'{customers:,}'                        # → '7,043'
f'{customers:08d}'                      # → '00007043' (zero-padded to 8 digits)

# Expressions inside braces
f'High risk: {high_risk_n / total * 100:.1f}%'
```

✗ Old string concatenation	✓ f-string (readable)
`msg = 'Customers: ' + str(total_cust) + ', Churn: ' + str(churn_rate) + '%'`	`msg = f'Customers: {total_cust:,}, Churn: {churn_rate:.1f}%'`

CH 03

Functions — Writing Reusable Logic

def, parameters, return, closures

Functions are the most important tool for keeping code organised and avoiding repetition. The Churn Platform defines more than 15 functions, each with a single clear purpose.

Anatomy of a Function

function_anatomy.py

```
# def    = keyword to start a function definition
# norm   = the function's name
# (s)    = parameter: the input the function receives
# →      = (conceptual arrow) the function produces a result
# return = keyword to send a result back to the caller

def norm(s):
    """Normalise a column name: lowercase, remove non-alphanumeric."""
    return re.sub(r'[^a-z0-9]', '', str(s).lower())

# Calling the function:
result = norm('Monthly Charges')    # result = 'monthlycharges'
result = norm('Customer_ID (No.)') # result = 'customeridno'
```

CONCEPT

The triple-quoted string immediately after 'def' is called a docstring. It describes what the function does. Type help(norm) in a Python shell to see it. Always write docstrings for functions others will use.

Default Arguments

Default arguments make a parameter optional. If the caller does not provide it, the default value is used.

default_args.py

```
def safe_float(v, default=0.0):   # 'default' is optional
    """Convert v to float. Return 'default' if conversion fails."""
    try:
        f = float(v)
        return f if np.isfinite(f) else default
    except Exception:
        return default

# All valid calls:
safe_float('42.5')          # → 42.5    (default 0.0 unused)
safe_float('bad text')      # → 0.0     (default used)
safe_float(float('inf'))    # → 0.0     (infinity → default)
```

```
safe_float('bad', -1.0)     # → -1.0    (caller overrides default)
```

MISTAKE

Never use a mutable object (list, dict) as a default value! Python creates the default only ONCE when the function is defined, so all calls share the same object.

def bad(items=[]): items.append(1) # Every call shares the SAME list!

def good(items=None): if items is None: items = [] # Create fresh each call

*args — Variable-Length Arguments

ANALOGY

Imagine a restaurant that normally serves a set meal for two. But you can call ahead and say 'we'll have 3 guests, or maybe 7'. *args lets your function accept 'as many as you want' arguments.

variable_args.py

```
def find_col(df, *keywords):
    """
    Find the first column matching any keyword.
    *keywords collects all extra arguments into a TUPLE.
    """
    cols_norm = {col: norm(col) for col in df.columns}
    for kw in keywords:           # iterate the tuple
        for col, n in cols_norm.items():
            if kw in n:
                return col
    return None

# Call with 1 keyword:
find_col(df, 'churn')

# Call with 8 keywords — same function handles both!
find_col(df, 'churn', 'attrition', 'churned', 'leave', 'exit', 'cancel',
'left', 'status')
```

**kwargs — Unpacking a Dictionary as Keyword Arguments

The ** operator unpacks a dictionary into keyword arguments. The Churn Platform uses this to pass classifier hyperparameters cleanly.

kwargs_unpacking.py

```
# The hyperparameters for each model are stored in dicts
rf_params = dict(n_estimators=300, max_depth=12, random_state=42, n_jobs=-1)

# **kw unpacks the dict: clf = RandomForestClassifier(n_estimators=300, ...)
clf = RandomForestClassifier(**rf_params)

# This is identical to writing it all out manually:
```

```
clf = RandomForestClassifier(n_estimators=300, max_depth=12, random_state=42,
n_jobs=-1)

# The advantage: you can swap models by changing the class, not the args
for Clf, name, kw in model_list:
    clf = Clf(**kw)    # works for RandomForest AND GradientBoosting
```

Closures — Inner Functions That Share State

A closure is an inner function that 'remembers' variables from its enclosing function, even after the outer function has returned. The Churn Platform uses this extensively inside build_report():

closures.py

```
def build_report(out_path):
    wb = writer.book   # 'wb' lives in build_report's scope
    F  = { ...formats... }

    def write_table(ws, df_t, row):   # inner function
        # 'wb' and 'F' are automatically available here!
        # No need to pass wb or F as arguments
        for col in df_t.columns:
            ws.write(row, col_index, col, F['col_hdr'])

    def add_chart(chart_type, src, ...):
        ch = wb.add_chart({'type': chart_type})  # wb available here too
        ...

    # Both inner functions share wb and F from their parent
    write_table(ws1, df_summary, 4)
    add_chart('column', sn_seg, 8, ...)
```

KEY POINT

Closures avoid passing the same argument (like 'wb' or 'F') to every single inner function. The inner function automatically 'closes over' variables from its parent scope. This is why they are called closures.

Lambda Functions — One-Line Anonymous Functions

A lambda is a small, nameless function defined in a single expression. Use them for simple transformations that are only needed once.

lambda_functions.py

```
# lambda <params>: <expression>

# Churn rate colour coding: green < 30%, amber < 60%, red >= 60%
cr_fmt = lambda val, alt: (
    F['c_red']   if float(val) > 0.6 else
```

```
    F['c_amber'] if float(val) > 0.3 else
    F['c_green']
)

# Equivalent named function:
def cr_fmt(val, alt):
    if   float(val) > 0.6: return F['c_red']
    elif float(val) > 0.3: return F['c_amber']
    else:                  return F['c_green']
```

CH 04

Data Structures — Organising Information

dict, list, set, frozenset, tuple, comprehensions

Python's built-in data structures are carefully chosen in the Churn Platform to match the access pattern required — lookups, ordered iteration, membership testing, and uniqueness.

Dictionaries { }

ANALOGY

A dictionary is a phone book. You already know the name (key) and want the number (value) instantly. You never flip through every entry — you jump straight to the name. Python dictionaries work exactly this way: any lookup is O(1) regardless of size.

dictionaries.py

```python
# Create
C = {
    'navy':  '#0B1F3A',
    'teal':  '#007C71',
    'red':   '#C0392B',
    'green': '#1E8449',
}

# Read
C['teal']           # → '#007C71'
C.get('missing')    # → None  (safe: no KeyError)
C.get('missing', '#000000')  # → '#000000'  (with default)

# Write / update
C['gold'] = '#F39C12'         # add new key
C.update({'amber': '#E67E22'}) # add/update multiple keys at once

# Check existence
'teal' in C         # → True
'pink' in C         # → False

# Iterate
for key, value in C.items():       # both key and value
    print(f'{key} = {value}')
```

frozenset — Immutable Sets for Fast Membership Testing

The Churn Platform maps churn labels like 'Yes', 'churned', 'left', 'inactive' to the number 1. It does this using a frozenset for O(1) lookup speed.

frozenset_usage.py

```
YES = frozenset({
    'yes', 'y', 'true', '1', '1.0',
    'churned', 'left', 'cancelled', 'cancel',
    'exit', 'quit', 'inactive', 'lost', 'gone',
})

NO = frozenset({
    'no', 'n', 'false', '0', '0.0',
    'active', 'retained', 'stay', 'current',
})

# Map a raw value to 0/1/NaN
raw = df['Churn'].astype(str).str.lower()
mapped = raw.map(lambda v:
    1 if v in YES else   # ← O(1) frozenset lookup
    0 if v in NO  else
    np.nan               # unknown values become NaN
)
```

Type	Properties	Used for
`list`	Ordered, mutable, allows duplicates	*FEATURES = ['tenure', 'monthly', ...]*
`tuple`	Ordered, IMMUTABLE, allows duplicates	*('utf-8', 'utf-8-sig', 'latin1')*
`set`	Unordered, mutable, NO duplicates	*seen = set(); seen.add(col)*
`frozenset`	Unordered, IMMUTABLE, NO duplicates	*YES = frozenset({'yes','y','1'})*
`dict`	Key→Value pairs, ordered (Python 3.7+)	*C = {'navy': '#0B1F3A'}*

Lists — When Order Matters

lists.py

```
FEATURES = ['tenure', 'MonthlyCharges', 'TotalCharges',
            'AvgMonthlySpend', 'ValueScore', 'LoyaltyScore', 'SpendVariance']

# Access by index (0-based)
FEATURES[0]    # → 'tenure'        (first item)
FEATURES[-1]   # → 'SpendVariance' (last item)
FEATURES[1:3]  # → ['MonthlyCharges', 'TotalCharges'] (slice)

# Add / remove
FEATURES.append('NewFeature')   # add to end
FEATURES.insert(0, 'First')     # insert at position 0
FEATURES.remove('tenure')       # remove by value
FEATURES.pop()                  # remove and return last item

# List operations
```

```
len(FEATURES)                # → 7   (count items)
'tenure' in FEATURES         # → True  (membership)
FEATURES.index('tenure')     # → 0    (position of item)
```

Comprehensions — Transforming Data in One Line

Comprehensions are one of Python's most elegant features. They build a new list or dictionary by applying an expression to every item in an existing sequence.

comprehensions.py

```
# ── LIST COMPREHENSION ──────────────────────────────────────
# Syntax: [expression  for item in iterable  if condition]

# Keep only columns that actually exist in the DataFrame
hr_cols = [c for c in desired_cols if c in df.columns]

# Square every number
squares = [x**2 for x in range(1, 6)]   # [1, 4, 9, 16, 25]

# Filter: keep only even numbers
evens = [x for x in range(10) if x % 2 == 0]  # [0, 2, 4, 6, 8]

# ── DICT COMPREHENSION ──────────────────────────────────────
# Syntax: {key_expr: value_expr  for item in iterable}

# Normalise every column name at once
cols_norm = {col: norm(col) for col in df.columns}
# → {'Monthly Charges': 'monthlycharges', 'Churn': 'churn', ...}

# ── SET COMPREHENSION ───────────────────────────────────────
unique_segs = {row['Segment'] for _, row in df.iterrows()}
```

✗ Traditional loop	✓ List comprehension

`# 4 lines, temporary variable, hard to read`
`col_widths = []`
`for col in df_src.columns:`
`  w = max(len(str(col)) + 2, 10)`
`  col_widths.append(min(w, 60))`
	`# 1 line, no temporary variable, immediately readable`
	`col_widths = [`
	`  min(max(len(str(col)) + 2, 10), 60)`
	`  for col in df_src.columns`
	`]`

CH 05

Control Flow — Decisions & Loops

if/elif/else, for, while, break, continue, match

Control flow determines the order in which statements execute. Without it, every programme would run the same way every time, which makes it useless for real-world problems.

if / elif / else — Making Decisions

Python uses indentation to show which code belongs inside a branch. There are no curly braces {} — the indentation IS the structure.

churn_mapping.py

```
if churn_col and churn_col in df.columns:
    # Branch 1: We found a real churn column
    raw_c  = df[churn_col].astype(str).str.strip().str.lower()
    mapped = raw_c.map(lambda v: 1 if v in YES else 0 if v in NO else np.nan)

    null_frac = mapped.isna().mean()    # what fraction could not be mapped?

    if null_frac > 0.60:
        # Sub-branch: too many unmapped values
        df['Churn']  = np.where(df['tenure'] < df['tenure'].median(), 1, 0)
        churn_source = f"Synthetic ('{churn_col}' had too many unknowns)"

    else:
        # Sub-branch: good enough — use real labels
        mode_v       = int(mapped.mode().iloc[0])
        df['Churn']  = mapped.fillna(mode_v).astype(int)
        churn_source = f"Detected: '{churn_col}'"

else:
    # Branch 2: no churn column found — generate synthetic labels
    df['Churn']  = (risk_score >= threshold).astype(int)
    churn_source = 'Synthetic — percentile risk score'
```

The Ternary Expression

A ternary expression is a one-line if/else. It is perfect for choosing between two values based on a condition.

ternary.py

```
# Syntax: value_if_true  if  condition  else  value_if_false

# Fill with median if valid, otherwise use default
fill = med if (pd.notna(med) and np.isfinite(med)) else float(default_val)

```

```
# Choose format based on row index (alternating colours)
fmt = F['cell_alt'] if (row_index % 2 == 1) else F['cell']

# Chained ternary (readable because values are short)
label = 'High' if score >= 0.60 else 'Med' if score >= 0.30 else 'Low'
```

for Loops — Iterating Over Sequences

Python's for loop iterates over any iterable: lists, dictionaries, DataFrames, ranges, and more.

for_loops.py

```
# ── Iterate a list ──────────────────────────────────────────
for col in df.columns:
    print(col)

# ── enumerate() — get index AND value ──────────────────────
for i, col in enumerate(df.columns):
    ws.write(0, i, col, header_fmt)   # write each header cell

# ── zip() — iterate two sequences in parallel ──────────────
for col, width in zip(df.columns, col_widths):
    ws.set_column(i, i, width)

# ── dict.items() — get key AND value ────────────────────────
for role, col in [('churn', churn_col), ('tenure', tenure_col)]:
    if col and col not in detected.values():
        detected[role] = col

# ── Nested loops: 6 encodings × 6 separators = 36 attempts ──
for enc in ENCODINGS:
    for sep in SEPARATORS:
        df = _try_csv(path, enc, sep)
        if df is not None:
            return df   # stop as soon as we succeed
```

break, continue, else on loops

break_continue.py

```
for Clf, name, kw in model_list:
    try:
        clf = Clf(**kw)
        clf.fit(X_train, y_train)
        break   # ← SUCCESS: exit loop, don't try other models
    except Exception:
        continue  # ← FAIL: skip to next model in the list

# The 'else' clause on a loop runs ONLY if break was never triggered
```

```
for attempt in range(MAX_RETRIES):
    if try_save(): break
else:
    raise RuntimeError('All retries exhausted')
```

Keyword	Effect	Used when...
`break`	Exit the loop immediately	*Stop after first successful model*
`continue`	Skip rest of this iteration	*Skip a failed model, try next*
`pass`	Do nothing (syntactic placeholder)	*Empty except block: except E: pass*
`else`	Run if loop completed without break	*Run fallback after all retries fail*

while Loops

while loops repeat as long as a condition is True. The Churn Platform uses the for+break pattern instead, but here is how while works conceptually:

while_loop.py

```
attempt = 0
while attempt < MAX_RETRIES:
    try:
        save_file(path)
        break   # success — exit
    except PermissionError:
        attempt += 1
        time.sleep(0.5)
```

CH 06

Strings & Regular Expressions

Text processing, pattern matching, clean_num()

The Churn Platform processes raw data from the real world. Real-world data is messy: column names have spaces and special characters, numbers contain currency symbols and commas, churn labels might be 'Yes', 'YES', 'yes', '1', or 'churned'. Python's string tools and the re module handle all of this.

Essential String Methods

string_methods.py

```
s = '  ₹ 1,23,456.78  '

s.strip()                # → '₹ 1,23,456.78'  (remove leading/trailing spaces)
s.strip().lower()        # → '₹ 1,23,456.78'  (to lowercase)
s.strip().upper()        # → '₹ 1,23,456.78'  (to uppercase)
s.replace('₹', '')       # → '  1,23,456.78  ' (replace substring)
s.split(',')             # → ['  ₹ 1', '23', '456.78  '] (split into list)
'hello world'.startswith('hello')  # → True
'hello world'.endswith('world')    # → True
'hello world'.count('l')           # → 3 (count occurrences)
'  hello  '.strip()                # → 'hello'
'Hello'.lower() == 'hello'         # → True (case-insensitive compare)
```

Method Chaining

String methods return a new string, so you can chain them left-to-right:

method_chaining.py

```
# Reading left to right: start → strip spaces → remove currency → remove inner spaces
s = (series
     .astype(str)                            # ensure it's text
     .str.strip()                            # trim whitespace
     .str.replace(r'[₹$€£¥]', '', regex=True)  # remove currency symbols
     .str.replace(r'\s+',    '', regex=True)   # remove all spaces
    )

# Equivalent to (much harder to read):
s = series.astype(str)
s = s.str.strip()
s = s.str.replace(r'[₹$€£¥]', '', regex=True)
s = s.str.replace(r'\s+',    '', regex=True)
```

Regular Expressions (the re module)

ANALOGY

Regex is like a very smart 'Find and Replace' in a text editor. Instead of searching for an exact word, you describe a pattern — 'any digit, followed by exactly three more digits, followed by a comma'. Regex finds ALL matches that fit that description.

Pattern	What it matches
`\d`	Any single digit: 0, 1, 2, … 9
`\d{3}`	Exactly 3 digits in a row
`\d+`	One OR MORE digits
`\d*`	Zero or more digits
`\d{1,3}`	Between 1 and 3 digits
`[^a-z0-9]`	Any character that is NOT a lowercase letter or digit
`[₹$€£¥]`	Any ONE of these specific currency symbols
`\s+`	One or more whitespace characters (space, tab, newline)
`.`	Any single character except newline
`^`	Start of string (when outside [])
`$`	End of string
`(,\d+)?`	Optional group: a comma followed by one or more digits

regex_functions.py

```python
import re

# re.sub(pattern, replacement, string) — find-and-replace
re.sub(r'[^a-z0-9]', '', 'Monthly Charges (Rs.)') # → 'monthlychargesrs'
re.sub(r'[₹$€£¥]',   '', '₹1,23,456')             # → '1,23,456'
re.sub(r'\s+',        ' ', 'too   many   spaces')  # → 'too many spaces'

# re.search(pattern, string) — find pattern anywhere in string
# Returns a Match object if found, None if not found
re.search(r'\d,\d{3}', '1,234.56')  # → Match object (US format found)
re.search(r'\d,\d{3}', '1234.56')   # → None (no match)

# re.fullmatch(pattern, string) — entire string must match
re.fullmatch(r'\d{1,3}(\.\d{3})+(,\d+)?', '1.234,56')  # European format
re.fullmatch(r'\d{1,3}(\.\d{3})+(,\d+)?', '1234.56')   # → None (no match)
```

Deep Dive: clean_num() — The Currency Converter

The clean_num() function is a masterpiece of defensive string handling. It converts columns that might contain '₹1,23,456' or '1.234,56' or '$ 99.99' into proper Python floats.

clean_num.py

```python
def clean_num(series):
    # Step 1: Remove symbols and whitespace
    s = (series.astype(str)
                .str.strip()
                .str.replace(r'[₹$€£¥]', '', regex=True)   # currency
                .str.replace(r'\s+', '', regex=True))       # spaces

    def _fix(v):
        # Step 2: Detect locale and normalise decimal separator

        # European format: 1.234,56 → 1234.56
        # Pattern: digit . groups of 3 digits , decimal part (optional)
        if re.fullmatch(r'\d{1,3}(\.\d{3})+(,\d+)?', v):
            return v.replace('.', '').replace(',', '.')  # remove . then , → .

        # US format: 1,234.56 → 1234.56
        # Pattern: a comma followed by exactly 3 digits then . or ,
        if re.search(r'\d,\d{3}[.,]', v):
            return v.replace(',', '')   # just remove the thousands comma

        # Indian format: 1,23,456.78
        # Pattern: digit , two digits , three digits
        if re.search(r'\d,\d{2},\d{3}', v):
            return v.replace(',', '')

        return v.replace(',', '')   # fallback: remove all commas

    s = s.apply(_fix)   # apply _fix to every value in the Series

    # Step 3: Remove any remaining non-numeric characters
    s = s.str.replace(r'[^\d.\-]', '', regex=True)

    # Step 4: Convert to float; non-parseable values become NaN
    return pd.to_numeric(s, errors='coerce')
```

CH 07

Error Handling — Writing Uncrashable Code

try/except/finally, defensive patterns, retry loops

Production code runs on data you did not write, in environments you did not configure, on machines you do not control. Error handling is what separates a script that works once from a tool that works every time.

What is an Exception?

ANALOGY

An exception is Python's way of raising its hand and saying: 'I cannot continue. Something unexpected happened.' Without a try/except, your entire programme stops. With one, you catch the problem and decide how to respond.

exception_examples.py

```
# These lines would CRASH without error handling:
int('hello')              # ValueError: invalid literal
1 / 0                     # ZeroDivisionError
d = {}; d['key']          # KeyError: 'key'
open('/nonexistent')      # FileNotFoundError
x = None; x.upper()      # AttributeError: NoneType has no .upper
```

The try / except / else / finally Structure

try_except_structure.py

```
try:
    # Code that MIGHT raise an exception
    df = pd.read_csv(path, encoding=enc)

except FileNotFoundError:
    # Runs ONLY if FileNotFoundError was raised
    print(f'File not found: {path}')

except (ValueError, UnicodeDecodeError) as e:
    # Catch MULTIPLE exception types in one clause
    print(f'Parsing failed: {e}')

except Exception as e:
    # Catch EVERYTHING ELSE — the final safety net
    # 'e' is the exception object; str(e) gives the message
    print(f'Unexpected error: {e}')

else:
    # Runs ONLY if NO exception occurred
    print('File loaded successfully!')
```

```

finally:
    # ALWAYS runs — even if an exception was raised and not caught
    # Use for cleanup: close files, destroy GUI windows, etc.
    root.destroy()
```

Python's Exception Hierarchy

All built-in exceptions inherit from BaseException. The most common ones inherit from Exception:

exception_hierarchy.py

```
BaseException
   ├── SystemExit          ← sys.exit() raises this (not a real error)
   ├── KeyboardInterrupt   ← user pressed Ctrl+C
   └── Exception           ← parent of all 'real' errors
         ├── ValueError    ← wrong value type  int('hello')
         ├── TypeError     ← wrong type  1 + 'hello'
         ├── KeyError      ← dict key missing  d['bad_key']
         ├── IndexError    ← list index out of range  x[999]
         ├── AttributeError ← object has no that attr  None.upper()
         ├── ZeroDivisionError ← divided by zero  1/0
         ├── FileNotFoundError ← file does not exist
         ├── PermissionError   ← no permission to open/write file
         └── RuntimeError      ← general runtime failure (used by this code)
```

TIP

Always catch the MOST SPECIFIC exception you expect first, and 'except Exception' last. This way, each type of failure gets the most appropriate response.

Never use 'except:' with no type — it catches SystemExit and KeyboardInterrupt too, which stops the user from quitting the programme.

The Retry Loop Pattern

The safe_write_excel() function implements one of the most useful real-world patterns: try something, and if it fails temporarily, wait and try again up to a maximum number of times.

retry_loop.py

```
def safe_write_excel(writer_fn, save_path):
    attempt_path = save_path

    for attempt in range(1, MAX_RETRIES + 1):   # attempts: 1, 2, 3
        try:
            writer_fn(attempt_path)   # try to write the file
            return attempt_path      # ← success: exit function

        except PermissionError:
            # File is open in Excel — offer the user two choices
```

```
            use_auto = ask_yesno(
                'File Locked',
                'File is open in another programme.\n'
                'Yes → auto-save with timestamp\n'
                'No  → choose a different location'
            )
            if use_auto:
                attempt_path = _timestamped(save_path)   # add HH-MM-SS
            else:
                attempt_path = save_file(os.path.basename(save_path))

        except OSError as exc:
            if attempt == MAX_RETRIES:
                raise RuntimeError(f'Failed after {MAX_RETRIES} attempts: {exc}')
            time.sleep(0.5)   # wait 0.5s before next attempt

    raise RuntimeError('All retry attempts exhausted.')
```

Defensive Guard Patterns

Rather than waiting for an exception, defensive guards check for dangerous conditions BEFORE they become errors:

defensive_guards.py

```
# Guard 1: Prevent ZeroDivisionError
high_risk_pct = high_risk_n / max(total_cust, 1) * 100
#                              ↑ if total_cust=0, use 1 instead

# Guard 2: Guard against NaN before arithmetic
fill = med if (pd.notna(med) and np.isfinite(med)) else 0.0

# Guard 3: Guard against infinity in results
def safe_float(v, default=0.0):
    try:
        f = float(v)
        return f if np.isfinite(f) else default   # catch +inf, -inf
    except Exception:
        return default

# Guard 4: Replace None from cancelled dialog with empty string
file_path = ask_file() or ''   # None → ''
if not file_path: sys.exit(0)  # '' is falsy → exit cleanly

# Guard 5: Clip numeric columns to valid ranges
df['MonthlyCharges'] = df['MonthlyCharges'].clip(lower=0)  # no negative prices
```

REMEMBER	The principle behind all defensive guards: ASSUME the data is wrong and check before using it. If you assume data is always correct, the one time it is not will crash your programme.

CH 08

Pandas — Data Tables in Python

DataFrame, Series, cleaning, filtering, groupby

Pandas is the foundation of data work in Python. Its DataFrame is a table of data with named columns and row indices — like a spreadsheet in memory, but programmable.

DataFrame vs Series

ANALOGY

A DataFrame is a spreadsheet — rows and columns. A Series is a single column from that spreadsheet. Almost every Pandas operation returns one or the other.

dataframe_basics.py

```python
import pandas as pd

# Create a DataFrame from a CSV
df = pd.read_csv('customers.csv', dtype=str)   # read ALL as strings first

# Basic info
df.shape        # → (7043, 21)  (rows, columns)
df.columns      # → Index(['CustomerID', 'Churn', 'MonthlyCharges', ...])
df.dtypes       # → column types
df.head(3)      # → first 3 rows
df.tail(3)      # → last 3 rows
len(df)         # → 7043 (number of rows)

# Access a single column → returns a Series
churn_col = df['Churn']        # Series of 7043 values

# Access multiple columns → returns a DataFrame
subset = df[['Churn', 'MonthlyCharges', 'tenure']]
```

Cleaning Operations

cleaning.py

```python
# Remove rows where EVERY column is NaN
df = df.dropna(how='all')

# Remove COLUMNS where every row is NaN
df = df.dropna(axis=1, how='all')

# Remove duplicate rows
df = df.drop_duplicates()

# Reset row index to 0, 1, 2, ... after filtering
```

```
df = df.reset_index(drop=True)    # drop=True discards the old index

# Remove auto-generated 'Unnamed: 0' columns from CSV saves
df = df.loc[:, ~df.columns.str.match(r'^Unnamed')]
#  ↑ df.loc[all_rows, only_columns_where_NOT_unnamed]

# Trim whitespace from column names
df.columns = [str(c).strip() for c in df.columns]
```

Boolean Masking — The Core of Filtering

Boolean masking is the most powerful Pandas feature. You create a column of True/False values and use it to select rows.

boolean_masking.py

```
# Step 1: Create a boolean mask (a Series of True/False)
high_risk_mask = df['Churn_Prob'] >= 0.60
# → [True, False, True, True, False, True, ...]

# Step 2: Use the mask to filter rows
high_risk_df = df[high_risk_mask]

# Or combined in one line:
high_risk_df = df[df['Churn_Prob'] >= 0.60]

# Combine conditions: & = AND, | = OR, ~ = NOT
# (each condition MUST be in parentheses when combining)
hi_risk_hi_val = df[(df['Churn_Prob'] >= 0.60) & (df['Predicted_CLV'] >= median)]
low_risk       = df[~(df['Churn_Prob'] >= 0.60)]   # ~ inverts the mask

# Count how many rows match
high_risk_n = int((df['Churn_Prob'] >= 0.60).sum())   # True counts as 1

# df.loc[mask, column] — filter rows AND select a specific column
df.loc[bad_total_mask, 'TotalCharges'] = corrected_values
```

groupby — Summarise by Category

groupby splits your DataFrame into groups by a column's value, applies a function to each group, then combines the results into one summary table.

groupby.py

```
# Named aggregation: new_column_name = (source_column, function)
summary = df.groupby('Segment', as_index=False).agg(
    Customers     = ('Churn',          'count'),  # count rows per segment
    Churned       = ('Churn',          'sum'),    # sum of 1s (churned)
```

```
    Churn_Rate    = ('Churn',          'mean'),   # mean of 0s and 1s = rate
    Avg_CLV       = ('Predicted_CLV',  'mean'),
    Total_CLV     = ('Predicted_CLV',  'sum'),
    Avg_Tenure    = ('tenure',         'mean'),
    Avg_Monthly   = ('MonthlyCharges', 'mean'),
)

# Sort by churn rate descending
summary = summary.sort_values('Churn_Rate', ascending=False)
```

pd.cut() — Binning Numeric Data

pd.cut() divides a continuous numeric column into buckets (bins). This turns a number like 0.73 churn probability into a human-readable label like 'High Risk (60-100%)'.

pd_cut.py

```
# Create 3 bins from Churn_Prob
df['Risk_Tier'] = pd.cut(
    df['Churn_Prob'],
    bins=[0, 0.30, 0.60, 1.001],          # bin EDGES
    labels=[
        'Low Risk (0-30%)',
        'Medium Risk (30-60%)',
        'High Risk (60-100%)',
    ],
    include_lowest=True   # include 0.0 in the first bin
)

# create_histogram_bins for a column
cuts = pd.cut(series, bins=6, duplicates='drop')
histogram = cuts.value_counts().reset_index()
# → DataFrame with columns ['Range', 'Count']
```

CH 09

NumPy — Fast Numerical Computing

Vectorisation, np.where, np.select, math operations

NumPy is Python's numerical computing library. Its core idea is vectorisation: instead of writing a loop that processes one number at a time, you apply an operation to an entire array at once. This is typically 100–1000× faster.

Vectorisation — Why Loops Are Slow

ANALOGY

Imagine stamping 'APPROVED' on 10,000 forms. The loop approach: pick up one form, stamp it, put it down — 10,000 times. The NumPy approach: load all 10,000 forms into a machine that stamps them all simultaneously in one press. Same result, tiny fraction of the time.

✗ Python loop (slow)

```
# Python loop — slow for large data
result = []
for i in range(len(df)):
    avg = df['Total'].iloc[i] /
(df['tenure'].iloc[i] + 1)
    result.append(max(0, avg))
df['AvgMonthlySpend'] = result
```

✓ NumPy vectorised (fast)

```
# NumPy vectorisation — fast for any size

df['AvgMonthlySpend'] = (
    df['TotalCharges'] / (df['tenure'] + 1)
).clip(lower=0)
# All 7,000+ rows computed in one operation
```

Arithmetic & Math Functions

numpy_math.py

```
# Element-wise arithmetic — operates on every row
df['TotalCharges'] / (df['tenure'] + 1)    # divide every row
df['MonthlyCharges'] * 12                   # multiply every row
df['tenure'] ** 2                           # square every row

# np.log1p(x) = log(1 + x)  — natural log that's safe when x = 0
# Why +1? Because log(0) = -infinity (crashes!)
df['ValueScore'] = np.log1p(df['TotalCharges'])

# .clip(lower, upper) — constrain values to a range
df['AvgMonthlySpend'] = df['AvgMonthlySpend'].clip(lower=0)  # no negatives
df['Churn_Prob']      = df['Churn_Prob'].clip(0, 1)          # must be 0-1

# .rank(pct=True) — express each value as a percentile (0.0 to 1.0)
tenure_rank = df['tenure'].rank(pct=True)
# The customer with the longest tenure gets rank 1.0
# The customer with the shortest gets rank close to 0.0

# .rsub(1) — subtract FROM 1 (reverse subtraction)
inverted = tenure_rank.rsub(1)    # short tenure → close to 1.0 (high risk)
```

np.where() — Vectorised if/else

np_where.py

```python
# np.where(condition, value_if_true, value_if_false)

# Apply to every row simultaneously
df['Churn'] = np.where(
    df['tenure'] < df['tenure'].median(),
    1,     # short-tenure customers → probably churned
    0      # long-tenure customers  → probably retained
)

# Equivalent Python loop (much slower):
results = []
median = df['tenure'].median()
for tenure in df['tenure']:
    results.append(1 if tenure < median else 0)
df['Churn'] = results
```

np.select() — Vectorised if/elif/elif/else

np.select handles more than two choices. It is the vectorised version of an if/elif chain.

np_select.py

```python
# Define conditions as a list (checked in ORDER — first match wins)
conditions = [
    hi_risk & hi_clv,      # condition 1
    hi_risk & ~hi_clv,     # condition 2: hi_risk AND NOT hi_clv
    ~hi_risk & hi_clv,     # condition 3: NOT hi_risk AND hi_clv
    # Any row not matching 1, 2, or 3 goes to 'default'
]

choices = [                          # parallel list of values
    'High Risk - High Value',
    'High Risk - Low Value',
    'Low Risk - High Value',
]

df['Segment'] = np.select(
    condlist  = conditions,
    choicelist= choices,
    default   = 'Low Risk - Low Value'   # else branch
)
```

TIP

np.select conditions are checked in order. If a row matches condition 1 AND condition 2, it gets the value for condition 1 (first match wins). Structure your conditions from most specific to most general.

Boolean Operations on Arrays

boolean_arrays.py

```python
hi_risk = df['Churn_Prob'] >= 0.60    # Series of True/False
hi_clv  = df['Predicted_CLV'] >= median

# & = element-wise AND (not 'and'!)
hi_risk & hi_clv      # True only where BOTH are True

# | = element-wise OR  (not 'or'!)
hi_risk | hi_clv      # True where EITHER is True

# ~ = element-wise NOT (not 'not'!)
~hi_risk              # True where Churn_Prob < 0.60

# Count Trues: True == 1 in Python
high_risk_count = hi_risk.sum()        # → e.g. 1869
high_risk_pct   = hi_risk.mean() * 100 # → e.g. 26.5 (percentage)

# np.isfinite — guard against inf and NaN
np.isfinite(float('inf'))   # → False
np.isfinite(float('nan'))   # → False
np.isfinite(42.5)           # → True
```

CH 10

Machine Learning — Plain English

train/test split, Random Forest, probabilities, evaluation

Machine learning sounds intimidating, but the code that does it fits on one screen. The complexity is inside the library — scikit-learn handles the maths. Your job is to prepare the data, choose a model, train it, and measure it.

The 5-Step ML Workflow

Step	Name	Code
1	**Prepare data**	`X = df[FEATURES]; y = df['Churn']`
2	**Impute missing**	`imputer.fit_transform(X)`
3	**Split**	`train_test_split(X, y, test_size=0.20)`
4	**Train**	`clf.fit(X_train, y_train)`
5	**Evaluate**	`accuracy_score(y_test, clf.predict(X_test))`

Features vs Labels

Every ML problem has inputs (features) and a target (labels). The model learns to predict labels from features.

features_labels.py

```python
# FEATURES — input columns the model learns from
FEATURES = [
    'tenure',           # how long they've been a customer
    'MonthlyCharges',   # what they pay per month
    'TotalCharges',     # total spend ever
    'AvgMonthlySpend',  # derived: total / tenure
    'ValueScore',       # derived: log(total)
    'LoyaltyScore',     # derived: tenure × spend
    'SpendVariance',    # derived: how much monthly spend varies
]

X = df[FEATURES]    # feature matrix — shape: (7043, 7)

# LABEL — what we want to predict
y = df['Churn']     # target vector — shape: (7043,) — values: 0 or 1
```

train_test_split — Why We Need a Test Set

ANALOGY

Imagine teaching a student with practice questions, then testing them on the SAME questions. They might have memorised the answers, not truly understood the material. A real exam uses NEW questions. The test set is the new exam questions — data the model has never seen.

train_test_split.py

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,                # all features
    y,                # all labels
    test_size=0.20,   # 20% held out for evaluation
    random_state=42,  # fixed 'seed' → same split every run
    stratify=y        # keep same churn ratio in train and test
)

# Result sizes (with 7043 rows):
# X_train: 5634 rows → used to TEACH the model
# X_test:  1409 rows → used to TEST the model on unseen data
```

Random Forest — How It Works

random_forest.py

```python
from sklearn.ensemble import RandomForestClassifier

clf = RandomForestClassifier(
    n_estimators=300,         # grow 300 decision trees
    max_depth=12,             # each tree can be at most 12 levels deep
    class_weight='balanced',  # compensate for fewer churned customers
    random_state=42,          # reproducibility
    n_jobs=-1                 # use ALL available CPU cores
)

# TRAINING — the model learns from historical data
clf.fit(X_train, y_train)

# PREDICTION — get churn PROBABILITY for each customer
# predict_proba returns [[P(no_churn), P(churn)], ...]
# We want column 1 (index [:, 1]) = probability of churning
df['Churn_Prob'] = clf.predict_proba(X)[:, 1]
```

Evaluation Metrics

Metric	Explanation
`accuracy_score`	% of predictions correct. Simple but misleading when classes are imbalanced (e.g. 90% not-churned → predict all 'no' = 90% accuracy!)

`roc_auc_score`	Area Under ROC Curve. Ranges 0.5 (random guessing) to 1.0 (perfect). More robust than accuracy for imbalanced data.
`predict_proba()`	Returns probability [P(class=0), P(class=1)] per row. Better than hard yes/no — lets you set your own risk threshold.

CAUTION

If churn labels are SYNTHETIC (invented by a formula), accuracy and AUC are meaningless — they only measure how well the model memorised your formula. The Churn Platform detects this and suppresses both metrics, showing 'N/A — Synthetic' instead.

CH 11

Feature Engineering

Turning raw columns into predictive signals

Raw data columns are rarely in the perfect form for a machine learning model. Feature engineering creates new, more informative columns from the raw ones. These derived columns capture business meaning that the model can learn from.

The 4 Engineered Features

feature_engineering.py

```python
# ── 1. Average Monthly Spend ──────────────────────────────
# How much does this customer spend per month on average?
# +1 prevents division by zero for brand-new customers (tenure=0)
df['AvgMonthlySpend'] = (df['TotalCharges'] / (df['tenure'] + 1)).clip(lower=0)

# ── 2. Value Score ────────────────────────────────────────
# Log-transform of TotalCharges to reduce skew from outliers
# log1p(x) = log(x+1) — safe when x=0, log(0) would be -infinity
df['ValueScore'] = np.log1p(df['TotalCharges']).clip(lower=0)

# ── 3. Loyalty Score ──────────────────────────────────────
# High tenure AND high total spend → very loyal customer
# MonthlyCharges+1 in denominator prevents division-by-zero
df['LoyaltyScore'] = (
    df['tenure'] * (df['TotalCharges'] / (df['MonthlyCharges'] + 1))
).clip(lower=0)

# ── 4. Spend Variance ─────────────────────────────────────
# Has this customer's monthly spend changed recently?
# Large variance = erratic billing = potential churn signal
df['SpendVariance'] = (
    (df['MonthlyCharges'] - df['AvgMonthlySpend'])
    / (df['AvgMonthlySpend'] + 1)  # normalise
).abs().clip(lower=0)
```

Why +1 Appears Everywhere (Laplace Smoothing)

You will notice '+1' added to denominators throughout the feature engineering code. This is not random — it prevents a specific type of crash called ZeroDivisionError.

laplace_smoothing.py

```python
# A brand-new customer has tenure = 0
# Without +1:
TotalCharges / tenure          # → ZeroDivisionError!
```

```

# With +1 (Laplace smoothing):
TotalCharges / (tenure + 1)     # tenure=0 → divide by 1 → safe
                                # tenure=12 → divide by 13 → slight nudge

# The error introduced by +1 is trivial for customers with real tenure
# The protection it gives for new customers (tenure=0) is essential
```

KEY POINT

Adding +1 to denominators is called Laplace smoothing or additive smoothing. It is used throughout statistics and machine learning to handle the 'zero division' problem gracefully. The small distortion it introduces is always worth the protection it provides.

The Synthetic Churn Risk Score

When no real churn column is found, the code generates synthetic churn labels using a weighted percentile formula. Understanding this formula teaches you how business rules get expressed in code:

synthetic_risk_score.py

```
# .rank(pct=True) gives each value a percentile rank from 0.0 to 1.0
# .rsub(1) = 1 - rank (inverts it: highest becomes lowest)

# Business reasoning for each weight:
# • Short tenure (0.40): new customers churn more than loyal ones
# • High monthly charge (0.30): expensive = more likely to shop around
# • High spend variance (0.20): erratic billing = dissatisfied customer
# • Low total charges (0.10): small total spend = less committed

risk_score = (
    df['tenure'].rank(pct=True).rsub(1) * 0.40   # short tenure → high risk
  + df['MonthlyCharges'].rank(pct=True)  * 0.30  # high charge → high risk
  + spend_variance.rank(pct=True)        * 0.20  # high variance → high risk
  + df['TotalCharges'].rank(pct=True).rsub(1) * 0.10  # low total → high risk
)

# Top 35% by risk score → labelled as Churned
threshold    = risk_score.quantile(0.65)    # 65th percentile
df['Churn']  = (risk_score >= threshold).astype(int)
```

CH 12 Best Practices & Design Patterns

DRY, naming, comments, SSOT, Factory, Strategy

The Churn Platform is not just a working script — it is well-engineered code. This chapter explains the design decisions that make it maintainable, readable, and extensible.

DRY — Don't Repeat Yourself

The most fundamental principle in software engineering: every piece of knowledge should have a single, authoritative place in the code. Duplication creates bugs — when you change the logic in one place, you forget to change it in the five other places you copy-pasted.

✗ Repeated code

```
# Writing the same format creation 6 times
hdr1 = wb.add_format({'bold': True, 'size': 10,
        'color': '#FFFFFF', 'bg': '#0B1F3A'})
hdr2 = wb.add_format({'bold': True, 'size': 10,
        'color': '#FFFFFF', 'bg': '#0B1F3A'})
# → Change one → must update all of them
```

✓ DRY with factory function

```
# Factory function: create once, call many times
def MF(bold=False, sz=10, fc='#0B1F3A', bg=None):
    return wb.add_format({...})

# One change in MF() updates every format
F = {'col_hdr': MF(bold=True, fc='#FFF', bg='#0B1F3A')}
```

Single Source of Truth (SSOT)

Every configurable value appears exactly once, at the top of the file. This is the constant pattern extended to its logical conclusion.

ssot.py

```
# The ONLY place where 31 appears
SHEET_NAME_LEN = 31

# Used via the constant in all other places
def _trunc(s, n=SHEET_NAME_LEN):
    return str(s)[:n]

# If Excel ever changes its limit to 32, you change it in ONE place
# and the entire codebase updates automatically
```

The Factory Method Pattern

A factory function creates objects based on parameters. MF() is a format factory — call it with any combination of parameters to get a tailored format object, without repeating the boilerplate.

factory_method.py

```python
def MF(bold=False, italic=False, sz=10, fc='#0B1F3A',
       bg=None, align='center', border=1, num_fmt=None):
    """Format factory — creates an XlsxWriter format object."""
    d = { 'font_name': 'Calibri', 'font_size': sz,
          'bold': bold, 'italic': italic, 'font_color': fc }
    if bg:      d['bg_color']   = bg
    if num_fmt: d['num_format'] = num_fmt
    return wb.add_format(d)

# Build the entire format registry from this one function:
F = {
    'title':    MF(bold=True, sz=18, fc='#FFF', bg='#0B1F3A'),
    'col_hdr':  MF(bold=True, sz=10, fc='#FFF', bg='#1A3A5C'),
    'cell':     MF(sz=10, fc='#0B1F3A', bg='#FFFFFF'),
    'num':      MF(sz=10, fc='#0B1F3A', num_fmt='#,##0'),
    'pct2':     MF(sz=10, fc='#0B1F3A', num_fmt='0.00%'),
}
```

The Strategy Pattern

The Strategy pattern lets you swap algorithms without changing the code that uses them. The ML training loop tries multiple classifiers — the calling code is identical regardless of which classifier is used.

strategy_pattern.py

```python
# A list of (Classifier class, name, hyperparameters)
# Adding a new model = add one line to this list
model_list = [
    (RandomForestClassifier,    'Random Forest',    rf_params),
    (GradientBoostingClassifier,'Gradient Boosting',gb_params),
    # (XGBClassifier,           'XGBoost',          xgb_params),  ← easy to add
]

# The loop is identical regardless of which model is used:
for Clf, name, kw in model_list:
    try:
        clf = Clf(**kw)           # 'Clf' could be any classifier
        clf.fit(X_train, y_train) # same API for all sklearn models
        break
    except Exception:
        continue
```

Naming Conventions

Convention	Used for
`ALL_CAPS`	Constants: MAX_RETRIES, FEATURES, SHEET_NAME_LEN
`lower_snake_case`	Variables and functions: churn_rate, find_col(), safe_float()

`CamelCase`	Classes: RandomForestClassifier, SimpleImputer (sklearn convention)
`_leading_underscore`	Private / internal helper: _trunc(), _fix(), _try_csv(), _dlg()
`df`	DataFrames — universal Pandas convention
`ws`	Worksheet — universal XlsxWriter convention
`wb`	Workbook — universal XlsxWriter convention
`np`	NumPy — universal alias (import numpy as np)
`pd`	Pandas — universal alias (import pandas as pd)

Writing Great Comments

Comments explain WHY code is written a certain way — not what it does (the code already says that). The best comments prevent future confusion:

comment_quality.py

```
# BAD COMMENT — just restates the code (adds no value)
x = x.clip(lower=0)  # clip to 0

# GOOD COMMENT — explains WHY
# TotalCharges < MonthlyCharges is physically impossible.
# This happens when data entry errors record a one-month customer
# with total = monthly instead of total = monthly × tenure.
bad_total = df['TotalCharges'] < df['MonthlyCharges']
df.loc[bad_total, 'TotalCharges'] = df.loc[bad_total, 'tenure'] *
df.loc[bad_total, 'MonthlyCharges']

# GOOD COMMENT — warns about a non-obvious limitation
# CLV_proxy = TotalCharges × (1 - Churn_Prob)
# This is NOT a DCF (Discounted Cash Flow) CLV.
# It represents the historical spend expected to be retained.
# Use for RELATIVE RISK RANKING only, not absolute valuation.
```

Pattern	Where in the code	Why it matters
SSOT Constants	`MAX_RETRIES, FEATURES, LABEL_ENC_MAX`	*Change once → updates everywhere*
Factory Method	`MF() format factory`	*100s of format variants, 1 function*
Strategy Pattern	`model_list = [(RF,...), (GB,...)]`	*Add new ML model in one line*
Retry Loop	`safe_write_excel()`	*Handles locked files gracefully*
DRY Functions	`write_table(), add_chart()`	*One function per responsibility*
Closures	`Inner functions in build_report()`	*Share wb, F, C without passing them*
Defensive Guards	`safe_float(), clip(), or ''`	*Zero crashes on bad/empty data*

frozenset Lookup	`YES = frozenset({...})`	*O(1) membership test vs O(n) list*
Vectorisation	`df col / (df col + 1)`	*100× faster than Python loops*
Fallback Chain	`load_file() engine cascade`	*Handles any file format automatically*

You have finished the Python Learner's Guide.

Every concept on these pages was taken directly from one real, working codebase.
Go back to the source code - you will now recognise every pattern, every choice, every line.

Python Learner's Guide · Customer Churn Intelligence Platform